P2P-CD-01

Peer-to-Peer Capability Discovery with Trust Gates

Specification Draft — Version 0.3

Status: Work in Progress | Category: Proposed Standard

Abstract

This document specifies a protocol for capability discovery between peers over connected sessions, with access control gates built into the protocol core. The protocol defines a CBOR-based wire format for capability advertisement (the discovery manifest), a hash-driven session state machine, role-aware capability negotiation with mutual active-set confirmation, and a peer cache mechanism for efficient re-connection handling. Trust credential schemes and transport bindings are defined in appendices.

This protocol assumes that capability advertisement is public by default. Peers that require confidential capability sets SHOULD restrict discovery to authenticated transports.

1. Introduction

Capability discovery between peers is underspecified in existing P2P protocols. Protocols such as libp2p identify, XMPP service discovery, and WebRTC SDP each solve parts of the problem in incompatible ways, with access control treated as an application-layer concern. This produces interoperability failures and forces implementations to reinvent authorization logic.

This specification defines a minimal, interoperable protocol for capability discovery that treats access control as a first-class protocol primitive. The core design goals are:

Trust-gated: access control is enforced before capability activation, not after.

Asymmetry-aware: peers independently declare provide and consume postures; role matching is explicit.

Change-reactive: any change to a peer's capability state triggers re-exchange via a hash-driven state machine.

Minimal: the normative core defines the smallest surface required for interoperability; policy, credentials, and transport bindings are appendices.

1.1 Out of Scope

The following are explicitly out of scope for this specification:

Peer discovery transport (how peers become aware of each other)

Identity provider integration and credential issuance

Behavioral reputation and scoring

Cryptographic algorithm mandates (algorithm agility is required; specific algorithms are not mandated)

Application-level capability semantics beyond namespace grammar

2. Terminology

The key words MUST, MUST NOT, SHOULD, SHOULD NOT, and MAY in this document are to be interpreted as described in RFC 2119.

Peer: A node that implements this protocol and participates in sessions.

Session: A mutually authenticated connection between exactly two peers over which capability exchange occurs.

Capability: A named, versioned, role-tagged function that a peer can provide to or consume from other peers. Capabilities are the atomic unit of negotiation. Each capability is declared using the namespace grammar defined in §4.4.

Discovery Manifest: The complete set of capability declarations, control parameters, and metadata advertised by a peer in a single exchange message. The canonical wire encoding is CBOR (§5).

Personal Hash: A hash computed by a peer over the deterministic CBOR serialization of its own discovery manifest (§4.5, §5.2). Each peer owns and maintains exactly one

personal hash.

Classification: A group reference assigned to a peer that governs which capabilities and scope parameters are applied to that peer's session. Classification is resolved per-capability: a single peer MAY receive different classifications for different capabilities within the same session.

Scope: The set of constraints applied to a capability for a given peer. Scope parameters are registry-extensible; the core protocol defines rate_limit and ttl.

Trust Gate: A gating predicate evaluated per-capability against a peer's resolved classification before that capability is activated. A trust gate produces ALLOW or DENY.

PEER_VISIBLE: A pre-session disposition. The peer is known to exist. No session is implied and no protocol obligations arise.

ACTIVE: A session state in which at least one capability was mutually negotiated, confirmed, and activated.

NONE: A terminal session outcome in which no capabilities were negotiated. Always results in session close.

DENIED: A pre-session disposition in which a previously seen peer hash is known to produce NONE. Negotiation is skipped. No transport connection is opened.

Active Set: The set of capabilities that both peers have confirmed as activated for a given session. The active set is the intersection of both peers' CONFIRM messages (§7.2).

3. Peer Discovery

Peer discovery is the process by which a peer becomes PEER_VISIBLE to another. This specification does not define the discovery transport. Discovery MAY occur over any broadcast or unicast medium including but not limited to local wireless broadcast, multicast DNS, DHT lookup, or rendezvous server.

3.1 Discovery Broadcast

A discovery broadcast MUST include the peer_id and personal_hash fields. A discovery broadcast MAY include the full discovery manifest. If the manifest is omitted, the receiving peer knows the peer exists and has a hash fingerprint, but MUST initiate a session to obtain the full manifest.

Receipt of a valid discovery broadcast MUST cause the receiving peer to record the sender as PEER_VISIBLE and store the sender's peer_id, personal_hash, and last_seen timestamp in the peer cache (§9).

When the full manifest is included in the discovery broadcast, it enables a receiving peer to evaluate compatibility and apply auto-DENY before any handshake occurs, reducing unnecessary session overhead. It also enables application-level use cases where visibility of a peer's capability intent prior to negotiation is a feature.

When the full manifest is omitted (lightweight discovery), auto-deny based on hash comparison still functions (the hash is always present), but pre-session compatibility filtering is not possible. Implementations that require capability confidentiality prior to authentication SHOULD use lightweight discovery.

3.2 Discovery Is Not a Session

A peer becoming PEER_VISIBLE does not initiate capability negotiation. A peer MAY remain PEER_VISIBLE indefinitely without a session being established. No protocol obligations arise from PEER_VISIBLE state alone.

4. Discovery Manifest Format

The discovery manifest is the atomic unit of capability advertisement. It is transmitted during session establishment (OFFER messages) and, optionally, during discovery broadcasts. When transmitted, the manifest MUST be sent in full; partial or delta manifests are not permitted in this version of the protocol.

4.1 Top-Level Fields

The discovery manifest contains the following top-level fields:

```
discovery_manifest {
```

```
    protocol_version : <unsigned integer>
    peer_id          : <cryptographic identity handle (byte string)>
    sequence_num     : <monotonically increasing unsigned integer>
    capabilities     : [ capability_declaration, ... ]
    personal_hash    : <hash of manifest state (byte string)>
    hash_algorithm   : <algorithm identifier (text string)>
}
```

All fields listed above are MUST-implement. A manifest missing any field MUST be rejected.

The protocol_version field identifies the version of this specification. For this document, the value MUST be 1. A peer that receives a manifest with a protocol_version it does not support MUST reject the manifest and SHOULD send a CLOSE message indicating unsupported version. Future revisions of this specification will increment this value. Peers SHOULD NOT attempt to negotiate downward; version negotiation is not defined in this version of the protocol.

The sequence_num field MUST increment with every new manifest issued by a peer. A receiver that observes a sequence_num equal to or less than the last seen sequence_num from that peer MUST treat the manifest as a replay and discard it.

4.2 Capability Declaration

Each entry in the capabilities array is a capability declaration:

```
capability_declaration {
    name             : <namespace string (text string)>
    role             : PROVIDE | CONSUME | BOTH
    mutual           : <boolean, default false>
    classification   : <group_ref (text string) | credential_blob (byte string)>
    scope            : scope_params
}
```

```
scope_params {
    rate_limit       : <unsigned integer, requests per second, 0 = unlimited>
    ttl              : <unsigned integer, seconds, 0 = no expiry>
}
```

The role field MUST be present on every capability declaration.

The mutual field MUST be set to true for capabilities that are intended to activate under a BOTH + BOTH role match (§7.4). If omitted, the default value is false.

The classification field MAY be omitted. If omitted, the receiving peer MUST apply its local default classification policy. Classification is per-capability: a peer MAY present different credentials or group references for different capabilities within the same manifest. When the classification carries a cryptographic credential, it SHOULD be encoded as a COSE structure (§5.4) for consistency with the wire format.

The scope_params fields are registry-extensible. The core protocol defines rate_limit and ttl. Additional scope parameters MAY be registered in the P2P-CAPDISC Scope Parameter Registry (§11). Each capability definition (e.g., Appendix B) SHOULD declare which scope parameters are applicable; scope parameters that are not applicable to a given capability MUST be ignored by the receiving peer.

4.3 Role Semantics

The role field declares a behavioral contract for the capability. The following roles are defined:

PROVIDE: The peer commits to accept incoming requests for this capability from matched peers. A PROVIDE peer acts as the server for this capability. If a PROVIDE peer cannot fulfill a capability after activation (e.g., due to resource exhaustion), it MUST rebroadcast with the capability removed or its scope modified (§8).

CONSUME: The peer declares intent to initiate requests against this capability on matched peers. A CONSUME peer acts as the client for this capability.

BOTH: The peer commits to both accept incoming requests and initiate outgoing requests simultaneously. A BOTH peer acts as both server and client. Capabilities declared with role BOTH that are intended to activate under a BOTH + BOTH match MUST also set mutual: true.

The role field is a binding declaration, not merely a statement of intent. A peer that advertises a role for a capability MUST be prepared to fulfill that role upon activation.

4.4 Namespace Grammar

Capability names MUST conform to the following grammar:

```
name         :=  org '.' scope '.' capability '.' version
org          :=  label ( '.' label )*
label        :=  [a-z] [a-z0-9-]* [a-z0-9]
scope        :=  [a-z] [a-z0-9]*
capability   :=  [a-z] [a-z0-9]*
version      :=  [0-9]+
```

The org segment is a reverse-domain-style organizational prefix that prevents namespace collisions across independent implementations.

The namespace prefix "core" is RESERVED for capabilities defined by this specification and its normative extensions. Implementations MUST NOT register capabilities under "core.*" without registration in the P2P-CAPDISC Well-Known Capabilities Registry (§11).

Examples of conformant capability names: core.stream.relay.1, core.feed.paginated.1, core.heartbeat.liveness.1, com.example.media.transcode.2, org.freedesktop.desktop.share.1.

Version is a required component of the name and represents a breaking change to the capability's semantics or wire format. Two capability names that differ only in version (e.g., core.feed.paginated.1 and core.feed.paginated.2) MUST be treated as distinct capabilities for the purposes of intersection computation. Non-breaking changes SHOULD be handled through scope parameters or capability-specific extension fields, not version increments.

4.5 Personal Hash Computation

The personal hash is computed over the deterministic CBOR serialization (§5.2) of the peer's manifest state:

```
personal_hash = hash_algorithm;H(canonical_cbor(peer_id, sequence_num, sorted(cap
```

Capabilities MUST be sorted lexicographically by name before serialization and hashing. Implementations that do not sort capabilities will produce non-deterministic hashes and MUST be considered non-conformant.

The sequence_num MUST be included in the hash computation to ensure that replayed manifests with identical capability sets but stale sequence numbers

produce different hashes.

The hash input MUST be the deterministic CBOR encoding (§5.2) of the fields being hashed. This ensures that two conforming implementations with the same logical state always produce the same hash value.

The hash_algorithm field MUST be prepended in the manifest.

Note: This specification does not mandate a specific hash algorithm. Implementations SHOULD select from algorithms registered in the IANA Named Information Hash Algorithm Registry. The algorithm identifier in the manifest MUST use the IANA registered name. Implementations SHOULD support SHA-256 as a baseline for interoperability.

5. Wire Format

This section defines the canonical wire encoding for all protocol messages. A single mandatory encoding ensures that two independent implementations produce identical byte sequences for the same logical message, which is essential for the hash-driven state machine defined in §4.5.

5.1 Canonical Encoding: CBOR

All protocol messages — discovery broadcasts, OFFER messages, CONFIRM messages, and CLOSE messages — MUST be encoded using CBOR (Concise Binary Object Representation) as defined in STD 94 (RFC 8949).

CBOR is mandated for the following reasons specific to this protocol:

Native binary byte strings: peer identities, hash digests, and credential blobs are binary data. CBOR encodes these directly without base64 overhead or encoding ambiguity.

Deterministic serialization: the personal hash (§4.5) requires that two implementations produce identical byte sequences from the same logical manifest. CBOR's deterministic encoding profile (§5.2) provides this guarantee.

Self-describing: CBOR is schema-free. A decoder can parse any conforming message without prior knowledge of the capability definitions it contains. This is essential for

the registry-extensible scope parameters and decentralized namespace defined in this specification.

IETF alignment: CBOR is an Internet Standard (STD 94) with IANA-registered extension tags, a formal schema language (CDDL, RFC 8610), and a companion signing and encryption specification (COSE, RFC 9052). These integrate directly with the protocol's trust gate and classification mechanisms.

5.2 Deterministic Encoding Profile

All CBOR encodings produced by this protocol MUST conform to the CBOR Core Deterministic Encoding Requirements defined in RFC 8949 §4.2.1. In summary:

Integers MUST be encoded in the smallest possible number of bytes.

Map keys MUST be sorted in bytewise lexicographic order of their encoded form.

Indefinite-length items MUST NOT be used.

Floating-point values MUST use the smallest precision that preserves the value (float16 if sufficient, then float32, then float64). For this protocol, all scope parameters are integers; floating-point values SHOULD NOT appear in protocol messages.

These requirements ensure that two conforming implementations encoding the same logical manifest will produce byte-identical output. This is the foundation of the hash-driven state machine: if two manifests are logically equal, their deterministic CBOR encodings are identical, and therefore their personal hashes are identical.

5.3 CDDL Schema Definitions

The following schemas are defined using the Concise Data Definition Language (CDDL) as specified in RFC 8610. These schemas are normative. A message that does not conform to the applicable schema MUST be rejected.

5.3.1 Discovery Manifest

```
discovery-manifest = {
    1 : uint,                    ; protocol_version (MUST be 1)
    2 : bstr,                    ; peer_id
    3 : uint,                    ; sequence_num
```

```
    4 : [+ capability-declaration],  ; capabilities
    5 : bstr,                        ; personal_hash
    6 : tstr,                        ; hash_algorithm (IANA registered name)
}
```

Map keys are integers to minimize encoded size. Implementations MUST use the integer keys as specified; text-string keys are not permitted in the canonical encoding.

5.3.2 Capability Declaration

```
capability-declaration = {
    1 : tstr,                        ; name (namespace grammar, §4.4)
    2 : role,                        ; role
    ? 3 : bool,                      ; mutual (default: false)
    ? 4 : classification,            ; classification
    ? 5 : scope-params,              ; scope
}

role = &(
    provide: 1,
    consume: 2,
    both:    3,
)

classification = tstr / bstr     ; group_ref (tstr) or credential_blob (bstr)

scope-params = {
    ? 1 : uint,                      ; rate_limit (requests/sec, 0 = unlimited)
    ? 2 : uint,                      ; ttl (seconds, 0 = no expiry)
    * int => any,                    ; registry-extensible additional params
}
```

The scope-params map is open-ended: the core protocol defines keys 1 (rate_limit) and 2 (ttl). Additional keys are allocated via the P2P-CAPDISC Scope Parameter Registry (§11). A receiver that encounters an unrecognized scope parameter key MUST ignore it.

5.3.3 OFFER Message

```
offer-message = {
    1 : uint,                        ; message_type (MUST be 1 for OFFER)
```

```
    2 : discovery-manifest,      ; the sender's full manifest
}
```

5.3.4 CONFIRM Message

```
confirm-message = {
    1 : uint,                    ; message_type (MUST be 2 for CONFIRM)
    2 : bstr,                    ; personal_hash (from sender's preceding OFFER)
    3 : [* tstr],                ; active_set (sorted capability names)
    ? 4 : {* tstr => scope-params},  ; accepted_params (per-capability)
}
```

The active_set array contains the fully qualified capability names that the sending peer considers activated. Entries MUST be sorted lexicographically. An empty array indicates no capabilities were matched.

The accepted_params map is OPTIONAL. When present, it carries the scope parameters that the sending peer has accepted or counter-proposed for each activated capability, keyed by capability name. If accepted_params is omitted, the receiver MUST treat the scope parameters from the OFFER as accepted without modification. If accepted_params is present for a given capability and the values differ from the OFFER, the receiving peer MUST use the more restrictive value for each numeric parameter (the lower of the two rate_limit values; the shorter of the two ttl values). Non-numeric parameter reconciliation is capability-specific and MUST be defined in the capability's specification.

5.3.5 CLOSE Message

```
close-message = {
    1 : uint,                    ; message_type (MUST be 3 for CLOSE)
    2 : bstr,                    ; personal_hash (sender's current hash)
    ? 3 : uint,                  ; reason_code
}
```

Reason codes are registry-extensible. The following are defined by this specification:

```
reason-code = &(
    normal:              0,      ; orderly shutdown
```

```
    no-match:            1,      ; NONE outcome
    auth-failure:        2,      ; HANDSHAKE failed
    version-unsupported: 3,      ; protocol_version not supported
    timeout:             4,      ; exchange timeout
    error:               255,    ; unspecified error
)
```

5.3.6 Message Framing

The message_type field (key 1) in each message allows a receiver to dispatch on the first decoded field. The defined message types are:

```
message-type = &(
    offer:   1,
    confirm: 2,
    close:   3,
)
```

All messages are self-contained CBOR maps. When transmitted over a stream transport (TCP, QUIC stream, WebSocket), each message MUST be preceded by a 4-byte unsigned big-endian length prefix indicating the number of CBOR bytes that follow. Over datagram transports (UDP broadcast), each datagram contains exactly one message with no length prefix.

5.4 Credential Encoding with COSE

When the classification field in a capability declaration carries a cryptographic credential (as opposed to a plain group_ref text string), the credential SHOULD be encoded using COSE (RFC 9052). This ensures that credential verification uses the same binary encoding as the rest of the protocol, avoiding format conversion between CBOR manifests and JSON-based credential formats.

The following COSE structures are applicable:

COSE_Sign1 (CBOR tag 18): A single-signer signed credential. Suitable for JWTs converted to CWT (CBOR Web Token, RFC 8392) format and for signed classification assertions.

COSE_Mac0 (CBOR tag 17): A MAC-authenticated credential for symmetric-key environments.

COSE_Encrypt0 (CBOR tag 16): An encrypted credential for environments requiring classification confidentiality.

A receiving peer MUST verify the COSE structure before using the claims within it for classification resolution (§6.2). The verification key is determined by the receiving peer's local trust policy and is out of scope for this specification.

Implementations MAY also accept non-COSE credential formats (JWT, W3C Verifiable Credentials, Macaroons) as described in Appendix A. However, COSE is RECOMMENDED for new deployments to maintain a single serialization format throughout the protocol stack.

5.5 JSON Interoperability

For debugging, logging, and interoperability with web-based systems, implementations MAY support JSON (RFC 8259) as an alternative encoding for display and diagnostic purposes. However:

The canonical encoding for hash computation MUST be deterministic CBOR (§5.2). Implementations MUST NOT compute personal hashes over JSON-encoded data.

The canonical encoding for wire transmission MUST be CBOR. Implementations MUST NOT transmit JSON-encoded messages in protocol exchanges.

When JSON is used for diagnostic output, the CBOR diagnostic notation defined in RFC 8949 §8 is RECOMMENDED for fidelity. Implementations MAY also produce standard JSON by converting CBOR byte strings to base64url and CBOR integer map keys to their string equivalents.

6. Trust Gates and Peer Classification

Access control in this protocol is enforced through trust gates evaluated per-capability against peer classification. A trust gate is a predicate that takes a peer's resolved classification for a specific capability as input and produces ALLOW or DENY as output.

6.1 Built-in Tiers

Two built-in classification tiers are defined normatively:

UNRESTRICTED: The peer is allowed to activate all advertised capabilities with no scope constraints applied.

DENIED: The peer is not allowed to activate any capabilities. If all capabilities resolve to DENIED, the session MUST be closed.

All classification tiers between UNRESTRICTED and DENIED are implementation-defined. Their definition SHOULD be documented in a Peer Classification Profile as described in Appendix A.

6.2 Classification Authority

A peer's classification is determined by the receiving peer using local policy. A connecting peer MAY assert a classification in its capability declaration via the classification field as a hint or credential. The receiving peer MAY accept, override, or ignore this assertion.

Classification resolution MUST complete before capability activation. Because classification is per-capability, a single peer MAY be classified differently for different capabilities within the same session (e.g., UNRESTRICTED for core.heartbeat.liveness.1 but a restricted tier for core.stream.relay.1).

6.3 Trust Gate Evaluation

For each capability in the received discovery manifest, the receiving peer MUST evaluate a trust gate using the resolved classification for that capability before the capability is considered for the active set.

If trust gate evaluation produces DENY for all capabilities, the session outcome MUST be set to NONE and the session MUST transition to CLOSED.

If trust gate evaluation produces DENY for a subset of capabilities, the denied capabilities MUST be excluded from the peer's computed active set. The session outcome depends on the intersection of the remaining capabilities with the remote peer's declarations.

If re-evaluation following a rebroadcast (§8) results in a previously ALLOW capability receiving DENY, that capability MUST be removed from the active set before the CONFIRM for the new exchange is sent.

7. Session Lifecycle

7.1 Session States

The following states govern the lifecycle of a peer relationship. PEER_VISIBLE and DENIED are pre-session dispositions; no transport connection exists. HANDSHAKE, CAPABILITY_EXCHANGE, and ACTIVE are session states. NONE is a terminal session outcome that always leads to CLOSED.

PEER_VISIBLE: Peer is known. No session is active. No protocol obligations.

HANDSHAKE: Mutual authentication is in progress. No capability exchange occurs in this state.

CAPABILITY_EXCHANGE: Peers are exchanging discovery manifests and confirming the active set. Entry point after HANDSHAKE. Re-entry point after any hash change during ACTIVE.

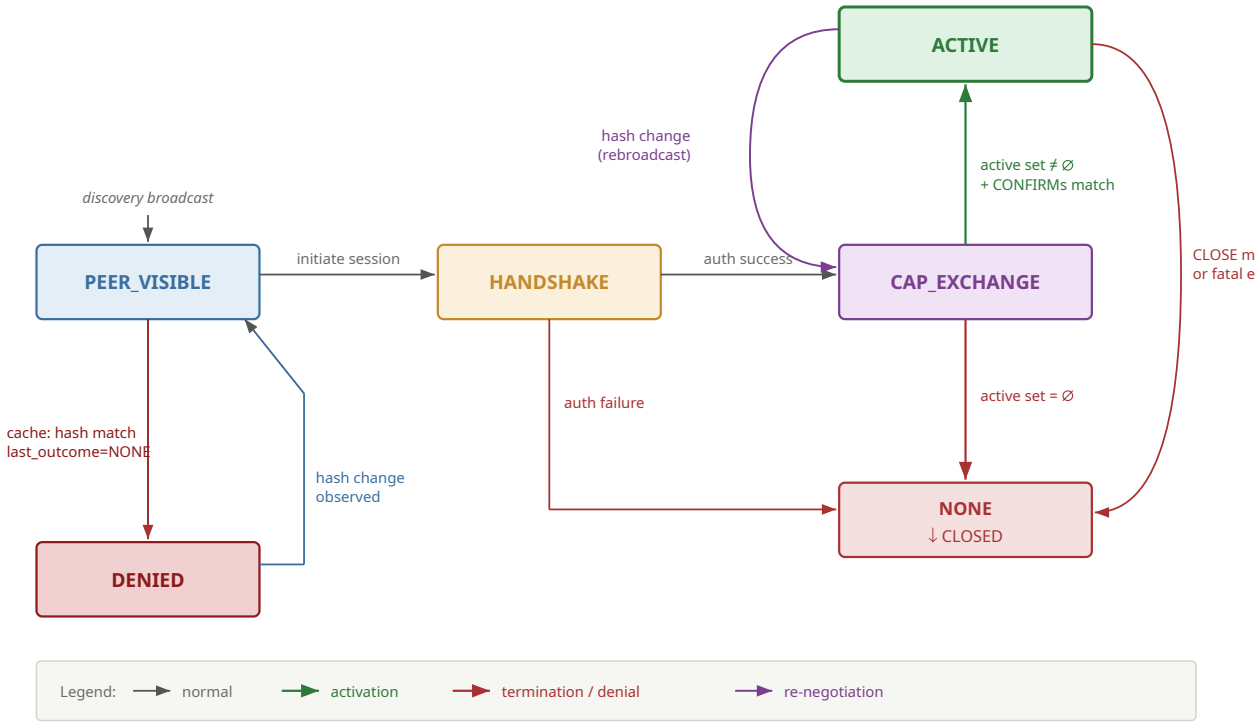
ACTIVE: At least one capability has been mutually negotiated, confirmed by both peers, and activated. Normal operating state.

NONE: No capabilities were negotiated or confirmed. Terminal outcome. Always results in session close.

DENIED: Pre-session disposition. Peer hash matches a cached NONE outcome. Negotiation is skipped. No transport connection is opened.

7.1.1 State Transition Diagram (Informational)

The following diagram is informational. The normative state transition table in §7.1.2 is authoritative where any discrepancy exists.



7.1.2 Normative State Transition Table

This table is the authoritative reference for session state transitions. Implementations MUST NOT allow transitions not listed here.

Current State	Event / Condition	Next State	Action(s)
`PEER_VISIBLE`	Local peer initiates connection	`HANDSHAKE`	Open transport connection. Begin mutual authentication per transport binding (Appendix C). If both peers simultaneously initiate, apply glare resolution (§7.1.3).
`PEER_VISIBLE`	Cache lookup: `last_hash == current_hash` AND `last_outcome == NONE`	`DENIED`	Record `outcome: DENIED` in peer cache. Do NOT open transport connection.
`DENIED`	New broadcast received with `hash ≠ last_hash`	`PEER_VISIBLE`	Update cache entry with new hash. Re-evaluate for session initiation.

Current State	Event / Condition	Next State	Action(s)
HANDSHAKE	Mutual authentication succeeds	CAP_EXCHANGE	Send OFFER message (§5.3.3) with full discovery manifest. Await remote OFFER.
HANDSHAKE	Authentication failure or timeout	CLOSED	Send CLOSE with reason_code 2. Tear down transport. Peer reverts to `PEER_VISIBLE` .
CAP_EXCHANGE	Both OFFERs and both CONFIRMs received; reconciled active set ≠ ∅	ACTIVE	Activate the reconciled active set with reconciled scope parameters. Begin normal operation.
CAP_EXCHANGE	Both OFFERs received; local intersection = ∅	NONE → CLOSED	Send CONFIRM with empty active set. Send CLOSE with reason_code 1. Record `last_outcome: NONE` in peer cache. Tear down transport.
CAP_EXCHANGE	CONFIRMs received; reconciled active set = ∅	NONE → CLOSED	Send CLOSE with reason_code 1. Record `last_outcome: NONE` in peer cache. Tear down transport.
CAP_EXCHANGE	CONFIRM contains personal_hash ≠ previously received OFFER hash	CAP_EXCHANGE	Remote state changed mid-exchange. Discard current exchange. Restart from OFFER. Increment local retry counter. If retry limit exceeded, send CLOSE with reason_code 4 and transition to CLOSED.
CAP_EXCHANGE	Timeout: remote OFFER or CONFIRM not received within implementation-defined window	CLOSED	Send CLOSE with reason_code 4 if transport allows. Tear down transport. Do NOT cache as NONE (timeout is not a negotiation outcome).
ACTIVE	Remote peer rebroadcasts with new personal_hash	CAP_EXCHANGE	Re-enter capability exchange. Existing active set remains operational until new exchange completes (§8.4).

Current State	Event / Condition	Next State	Action(s)
`ACTIVE`	Local capability state changes (any §8.1 trigger)	`CAP_EXCHANGE`	Recompute personal_hash, increment sequence_num, send new OFFER. Existing active set remains operational until re-exchange completes (§8.4).
`ACTIVE`	CLOSE message received from remote peer	`CLOSED`	Deactivate all capabilities. Tear down transport. Peer reverts to `PEER_VISIBLE` .
`ACTIVE`	Heartbeat failure (timeout exceeded)	`CAP_EXCHANGE`	Attempt re-exchange. If re-exchange produces NONE, transition to CLOSED.
`ACTIVE`	Ungraceful disconnect (transport error)	`CLOSED`	Deactivate all capabilities. Peer reverts to `PEER_VISIBLE` . Implementation MAY attempt reconnection.

7.1.3 Glare Resolution

If both peers simultaneously transition from PEER_VISIBLE to HANDSHAKE toward each other (simultaneous open), the peer with the lexicographically lower peer_id MUST abandon its outbound connection attempt and accept the inbound connection. The peer with the lexicographically higher peer_id proceeds with its outbound connection. This produces exactly one session per peer pair.

7.2 Capability Exchange — Four-Message Sequence

The following is the normative message sequence for capability exchange. This sequence is used both for initial exchange after a successful handshake and for re-exchange triggered by a hash change during ACTIVE state. All messages use the CBOR wire format defined in §5.

```
-- Phase 1: Capability Advertisement --
1.  A -> B : OFFER   [ manifest_A ]              (§5.3.3)
2.  B -> A : OFFER   [ manifest_B ]              (§5.3.3)
```

```
-- Phase 2: Active Set Confirmation --
    Each peer independently computes intersection + trust gate evaluation.
    Each peer sends its computed active set and accepted parameters.

3.  A -> B : CONFIRM [ hash_A, active_set_A, ?accepted_params_A ]  (§5.3.4)
4.  B -> A : CONFIRM [ hash_B, active_set_B, ?accepted_params_B ]  (§5.3.4)

-- Reconciliation --
    Each peer computes: final_set = active_set_A ∩ active_set_B
    For each capability in final_set:
        final_params = min(accepted_params_A, accepted_params_B)
    If final_set ≠ ∅  → both enter ACTIVE with final_set and final_params
    If final_set = ∅   → both enter NONE → CLOSED
```

Each peer MUST send its full discovery manifest in the OFFER message. Partial or delta offers are not permitted.

The active_set in the CONFIRM message is the sorted list of capability names that the sending peer considers activated after intersection computation and trust gate evaluation. Capabilities MUST be sorted lexicographically by name.

The personal_hash in each CONFIRM MUST match the personal_hash from that peer's preceding OFFER. If a peer receives a CONFIRM whose personal_hash differs from the hash in the OFFER previously received from that peer, the peer's state changed during the exchange. The receiving peer MUST discard the current exchange and restart from step 1. Implementations SHOULD enforce a retry limit; exceeding the retry limit MUST result in transition to CLOSED.

The reconciled active set (final_set) is the intersection of both CONFIRM active sets. If the reconciled set is empty, the session outcome MUST be NONE and the session MUST transition to CLOSED. If the reconciled set is non-empty, both peers enter ACTIVE with the reconciled set as the definitive active capabilities and the reconciled parameters as the definitive scope constraints.

7.2.1 Alternate Paths

Empty intersection at Phase 1: If a peer computes an empty intersection after receiving the remote OFFER (before sending CONFIRM), it MUST still send a CONFIRM with an empty active set. This allows the remote peer to distinguish "no match" from "timeout."

Hash change during exchange: Handled by the hash consistency check in the CONFIRM phase. See §7.2 above.

Timeout: If a peer does not receive the remote OFFER within an implementation-defined timeout, it MUST transition to CLOSED. If a peer does not receive the remote CONFIRM within an implementation-defined timeout after sending its own CONFIRM, it MUST transition to CLOSED. Timeouts MUST NOT be cached as NONE outcomes.

Simultaneous OFFER: Both peers may send OFFER before receiving the other's OFFER. This is the expected case and does not require special handling; the protocol is designed for concurrent OFFER transmission.

Protocol version mismatch: If a peer receives an OFFER with a protocol_version it does not support, it MUST send a CLOSE with reason_code 3 (version_unsupported) and transition to CLOSED.

7.3 Parameter Negotiation

Scope parameter negotiation occurs within the CONFIRM phase of the four-message exchange. Each peer advertises its offered parameters in the OFFER (via the scope field of each capability declaration). Each peer then declares its accepted parameters in the CONFIRM (via the optional accepted_params field).

Reconciliation follows a most-restrictive-wins rule for numeric parameters: the final value for rate_limit is the minimum of both peers' accepted values; the final value for ttl is the minimum of both peers' accepted values. This ensures that neither peer is committed to more than it offered.

For non-numeric parameters (e.g., codec, encoding), reconciliation rules MUST be defined by the capability's specification (Appendix B). If a capability's specification does not define reconciliation for a non-numeric parameter, the PROVIDE peer's value takes precedence.

If a peer determines that the reconciled parameters are unacceptable (e.g., the reconciled bitrate is below a usable threshold), it MAY exclude that capability from its active set in the CONFIRM, which will cause the capability to be absent from the reconciled final set.

7.4 Role-Based Intersection

Intersection is computed per capability using the role fields of both peers' declarations. The following rules define a match:

PROVIDE + CONSUME: Match. The PROVIDE peer acts as server; the CONSUME peer acts as client.

CONSUME + PROVIDE: Match. Identical to above from opposite perspective.

BOTH + PROVIDE: Match. The BOTH peer acts as client; the PROVIDE peer acts as server.

BOTH + CONSUME: Match. The BOTH peer acts as server; the CONSUME peer acts as client.

BOTH + BOTH: Match ONLY if both declarations have mutual: true. Both peers participate as simultaneous server and client. If either declaration has mutual: false or the field is absent, no match.

PROVIDE + PROVIDE: No match.

CONSUME + CONSUME: No match.

7.5 Connection Archetypes

Three connection archetypes are defined. Worked message sequence examples for each are provided in Appendix E.

NONE / NONE: Neither peer offers any capability the other can consume. Session outcome is NONE. Session closes.

ASYMMETRIC: One peer provides capabilities the other consumes. Session outcome is ACTIVE.

MUTUAL: Both peers provide capabilities to each other simultaneously. Both declare BOTH (with mutual: true) or complementary PROVIDE/CONSUME pairs. Session outcome is ACTIVE.

7.6 Session Termination

A session outcome of NONE MUST result in session close. Implementations MUST send a CLOSE message (§5.3.5) with the appropriate reason_code, tear down the transport connection, and record the outcome in the peer cache.

A closing peer SHOULD send a CLOSE message carrying its current personal hash and reason_code 0 (normal) before terminating the connection. Receipt of a CLOSE

message MUST cause the receiving peer to deactivate all capabilities and tear down the transport connection.

Ungraceful disconnection (no CLOSE message received) SHOULD be detected by the heartbeat mechanism (core.heartbeat.liveness.1, Appendix B). A heartbeat failure within the declared timeout constitutes a session failure and MUST trigger CAPABILITY_EXCHANGE re-entry. If re-exchange produces NONE, the session closes.

8. Change Detection and Rebroadcast

8.1 Rebroadcast Triggers

Any change to a peer's capability state MUST cause that peer to recompute its personal hash and rebroadcast its full discovery manifest. The following changes are normative triggers:

Capability addition: A new capability is added to the manifest. MUST rebroadcast.

Capability removal: An existing capability is removed. MUST rebroadcast.

Scope parameter change: Any scope parameter changes for any capability. MUST rebroadcast.

Classification change: The peer's own group membership changes. MUST rebroadcast immediately. All capabilities granted under the prior classification MUST be re-evaluated before the CONFIRM for the new exchange is sent.

Role change: A capability's role (PROVIDE/CONSUME/BOTH) changes. MUST rebroadcast.

8.2 Rebroadcast Behavior

Rebroadcast MUST transmit the full discovery manifest. Delta updates are not permitted in this version of the protocol.

The sequence_num field MUST be incremented before rebroadcast.

A peer that receives a discovery manifest whose personal hash differs from the last stored hash for that peer MUST re-enter CAPABILITY_EXCHANGE state.

8.3 Advisory Change Flags

A rebroadcast MAY include an advisory change_flags field indicating the nature of the change. This field is non-normative; receiving peers MUST still perform full re-evaluation regardless of its contents. Defined flag values are: ADDED, REMOVED, SCOPE_ONLY, ROLE_CHANGE, CLASSIFICATION_CHANGE. Implementations MAY use change_flags to optimize re-evaluation (e.g., fast-pathing scope-only changes) but MUST NOT rely on them for correctness.

8.4 Active Set Continuity During Re-exchange

When a session transitions from ACTIVE to CAPABILITY_EXCHANGE due to a hash change, the existing active set MUST remain operational until the new exchange completes. Capabilities in the current active set continue to function during re-negotiation. Once the new exchange produces a reconciled active set (§7.2), the new set replaces the prior set atomically. Capabilities present in the prior set but absent from the new set MUST be deactivated. Capabilities present in both sets MUST NOT be interrupted.

8.5 Re-evaluation Obligations

On receipt of a rebroadcast, the receiving peer MUST re-evaluate classification and scope for all capabilities in the new manifest.

If re-evaluation results in revocation of a previously ALLOW capability, revocation MUST be applied before the CONFIRM for the new exchange is sent.

9. Auto-Deny and Peer Cache

9.1 Peer Cache Structure

Each peer maintains a local peer cache. The cache records the outcome of past interactions indexed by peer identity and capability hash.

```
peer_cache_entry {
```

```
    peer_id          : bstr      ; cryptographic identity handle
    last_hash        : bstr      ; personal hash from last exchange
    last_outcome     : uint      ; ACTIVE=1 | NONE=2 | DENIED=3
    last_seen        : uint      ; timestamp (seconds since epoch)
}
```

Implementations that support auto-deny MUST store peer_id, last_hash, last_outcome, and last_seen for every session that terminates with outcome NONE. The last_seen field MUST be updated on every interaction with the peer, including discovery broadcasts.

The peer cache MUST NOT be transmitted to third parties.

The peer cache SHOULD be treated as sensitive local state and protected with equivalent controls to authentication credentials.

9.2 Auto-Deny Behavior

Auto-deny is RECOMMENDED. Implementations that support auto-deny MUST implement the following behavior: if a peer is PEER_VISIBLE and its current personal hash matches last_hash in the cache and last_outcome is NONE, the implementation MUST record outcome DENIED and MUST NOT initiate a handshake with that peer.

If auto-deny is implemented, the cache hash check MUST precede any handshake attempt.

If a peer's personal hash changes relative to last_hash in the cache, the implementation SHOULD attempt a new handshake regardless of last_outcome.

9.3 Reconnection on Capability Change

A peer whose manifest changes from containing no capabilities to containing one or more capabilities SHOULD attempt reconnection to known PEER_VISIBLE peers. A peer that adds capabilities to a non-empty manifest SHOULD rebroadcast to all active sessions per §8 and MAY attempt connection to PEER_VISIBLE peers whose cached last_outcome was NONE (since the hash has changed, auto-deny will not apply).

10. Security Considerations

10.1 Assumed Precondition

This protocol assumes a mutually authenticated session is established before capability exchange begins. Both peers MUST have verified each other's cryptographic identity handle prior to entering CAPABILITY_EXCHANGE state. The mechanism of mutual authentication is not defined here; implementations SHOULD use an established protocol such as TLS 1.3 mutual auth, the Noise Protocol Framework, or equivalent.

10.2 Replay Prevention

The sequence_num field in the discovery manifest is a monotonically increasing integer included in the personal hash computation. A receiver that observes a sequence_num not greater than the last seen value from that peer MUST discard the manifest as a potential replay.

10.3 Discovery Broadcast Exposure

When a full discovery manifest is included in a broadcast, the peer's capability set is visible to all receivers, including unauthenticated observers. This is a deliberate design trade-off enabling pre-session compatibility filtering. Implementations that require capability confidentiality prior to authentication MUST use lightweight discovery (§3.1) and restrict discovery to transports that provide channel-level encryption.

10.4 Peer Cache Privacy

The peer cache contains a record of observed peer identities, their capability hashes, and session outcomes. This constitutes a potentially sensitive log of peer interactions. Implementations MUST NOT transmit this data to third parties. Implementations SHOULD apply access controls to the cache equivalent to those applied to private key material.

10.5 Sybil Resistance

Sybil resistance is an identity-layer concern and is out of scope for this protocol. By requiring mutually authenticated sessions as a precondition, this protocol delegates

Sybil resistance to the identity layer. Implementations SHOULD select identity mechanisms with appropriate Sybil resistance properties for their deployment context.

10.6 Active Set Confirmation and Trust Asymmetry

The four-message exchange (§7.2) ensures that both peers explicitly confirm the active set before capability activation. However, because each peer applies its own local classification policy, the two peers may arrive at different active sets from the same pair of manifests. The CONFIRM reconciliation step resolves this by taking the intersection of both active sets, ensuring that no capability is activated unless both peers independently approved it. Implementations MUST NOT activate a capability that was not present in both CONFIRM messages.

10.7 Wire Format Security

CBOR decoders MUST enforce limits on nesting depth, map key count, and total message size to prevent resource exhaustion attacks. Implementations SHOULD reject messages exceeding implementation-defined size limits before full decoding. The deterministic encoding profile (§5.2) mitigates certain classes of canonicalization attacks that could cause hash mismatches between conforming implementations.

COSE-signed credentials (§5.4) MUST be verified before the claims within them are used for classification. Implementations MUST NOT trust the contents of a COSE structure whose signature or MAC verification has failed.

11. IANA Considerations

This document requests the following IANA registries:

P2P-CAPDISC Well-Known Capabilities Registry — maps capability name strings under the reserved "core.*" namespace to specification documents. Registration requires Standards Action.

P2P-CAPDISC Hash Algorithm Registry — maps algorithm identifier strings to hash algorithm specifications. Entries SHOULD reference algorithms registered in the IANA Named Information Hash Algorithm Registry.

P2P-CAPDISC Scope Parameter Registry — maps scope parameter integer keys to field names, type definitions, units, and applicability constraints. Keys 1 (rate_limit) and 2 (ttl) are registered by this document.

P2P-CAPDISC Message Type Registry — maps message type integer values to message names and specification sections. Values 1 (OFFER), 2 (CONFIRM), and 3 (CLOSE) are registered by this document.

P2P-CAPDISC Close Reason Code Registry — maps reason code integer values to reason names. Values 0–4 and 255 are registered by this document.

12. References

12.1 Normative References

RFC 2119 — Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, March 1997.

RFC 8446 — Rescorla, E., "The Transport Layer Security (TLS) Protocol Version 1.3", August 2018.

STD 94 / RFC 8949 — Bormann, C. and P. Hoffman, "Concise Binary Object Representation (CBOR)", December 2020.

RFC 8610 — Birkholz, H., Vigano, C., and C. Bormann, "Concise Data Definition Language (CDDL): A Notational Convention to Express Concise Binary Object Representation (CBOR) and JSON Data Structures", June 2019.

STD 96 / RFC 9052 — Schaad, J., "CBOR Object Signing and Encryption (COSE): Structures and Process", August 2022.

RFC 9053 — Schaad, J., "CBOR Object Signing and Encryption (COSE): Initial Algorithms", August 2022.

12.2 Informative References

RFC 8259 — Bray, T., Ed., "The JavaScript Object Notation (JSON) Data Interchange Format", STD 90, December 2017.

RFC 8392 — Jones, M., Wahlstroem, E., Erdtman, S., and H. Tschofenig, "CBOR Web Token (CWT)", May 2018.

RFC 8742 — Bormann, C., "Concise Binary Object Representation (CBOR) Sequences", February 2020.

RFC 9054 — Schaad, J., "CBOR Object Signing and Encryption (COSE): Hash Algorithms", August 2022.

Noise Protocol Framework — Perrin, T., "The Noise Protocol Framework", http://noiseprotocol.org/noise.html, July 2018.

W3C Verifiable Credentials Data Model — Sporny, M., et al., https://www.w3.org/TR/vc-data-model/

libp2p Identify Protocol — https://github.com/libp2p/specs/tree/master/identify

Appendix A. Trust Credential Formats

This appendix defines how common credential formats map to the classification field in the capability declaration. These mappings are informational. Implementations MAY support any subset. COSE-based credentials (§5.4) are RECOMMENDED for new deployments.

A.1 CBOR Web Token (CWT) via COSE

A CWT (RFC 8392) is the CBOR equivalent of a JWT and is the RECOMMENDED credential format for this protocol. The CWT is encoded as a COSE_Sign1 structure and placed in the classification field as a credential_blob (byte string). The receiving peer MUST verify the COSE signature before using the claims. The group reference is extracted from a designated claim. The CWT MUST include an exp claim; expired tokens MUST be rejected.

A.2 JSON Web Token (JWT)

A JWT may be placed in the classification field as a credential_blob. The JWT is encoded as a UTF-8 byte string within the CBOR message. The receiving peer MUST verify the JWT signature against a trusted public key before using the claims. Implementations SHOULD prefer CWT (A.1) over JWT for new deployments to avoid format conversion overhead.

A.3 W3C Verifiable Credentials

A W3C Verifiable Credential may be placed in the classification field. The receiving peer MUST verify the credential proof before accepting the credential subject claims as the group reference. This approach supports decentralized identity scenarios without a central credential issuer.

A.4 Local Policy Reference

The classification field MAY carry a group_ref text string that the receiving peer resolves against a local policy store. This is the simplest form of classification and requires no cryptographic verification of the field itself. The security of this approach depends entirely on the integrity of the authenticated session — the peer_id is trusted, and the local policy maps peer_id to group.

A.5 Macaroons

Macaroons may be used as credential_blobs where capability-scoped, delegatable authorization is required. The caveats in the macaroon encode the scope constraints directly, allowing the receiving peer to verify that the presented token authorizes specifically the capabilities being requested.

Appendix B. Reference Capability Definitions

This appendix defines reference capability implementations using the namespace grammar defined in §4.4. These definitions are normative for implementations that claim to support the named capabilities.

B.1 core.heartbeat.liveness.1

```
name:     core.heartbeat.liveness.1
role:     BOTH
mutual:   true
purpose:  Session liveness detection
params:   { interval_ms: uint, timeout_ms: uint }
scope:    ttl applies; rate_limit does not apply
```

Both peers MUST declare role BOTH with mutual: true for this capability to activate. One peer sends a PING message at interval_ms; the other MUST respond with PONG within timeout_ms. Failure to receive PONG within timeout_ms constitutes session failure and MUST trigger CAPABILITY_EXCHANGE re-entry.

PING and PONG are CBOR maps with message_type values 4 and 5 respectively:

```
ping-message = { 1 : 4, 2 : uint }   ; message_type=4, timestamp
pong-message = { 1 : 5, 2 : uint }   ; message_type=5, echo timestamp
```

B.2 core.stream.relay.1

```
name:     core.stream.relay.1
role:     PROVIDE | CONSUME
purpose:  Unidirectional continuous data stream
requires: transport.udp or transport.quic
params:   { bitrate_kbps: uint, codec: tstr }
scope:    ttl applies; rate_limit does not apply (continuous stream)
param_negotiation:
    bitrate_kbps: min(provider, consumer)
    codec: provider's value takes precedence
```

B.3 core.feed.paginated.1

```
name:     core.feed.paginated.1
role:     PROVIDE | CONSUME
purpose:  Paginated event or content feed
requires: transport.tcp or transport.quic
params:   { page_size: uint, encoding: tstr }
scope:    rate_limit applies (requests per second); ttl applies
param_negotiation:
    page_size: min(provider, consumer)
    encoding: provider's value takes precedence
```

Appendix C. Transport Bindings

This appendix defines how the capability exchange message sequence maps onto common transport protocols. Each binding is informational; implementations MAY define additional bindings. All messages use the CBOR wire format (§5) with 4-byte length-prefix framing over stream transports.

C.1 TCP

Discovery manifests are framed as length-prefixed CBOR messages over a TCP stream. Mutual authentication SHOULD be implemented using TLS 1.3 with client certificates. The HANDSHAKE state corresponds to the TLS handshake phase. CAPABILITY_EXCHANGE begins after the TLS handshake completes successfully.

C.2 QUIC

Discovery manifests are sent as length-prefixed CBOR messages on a dedicated QUIC stream (stream ID 0 by convention). QUIC's built-in mutual authentication via TLS 1.3 satisfies the handshake precondition. CAPABILITY_EXCHANGE begins on the first application-level datagram after QUIC connection establishment.

C.3 WebSockets

Discovery manifests are transmitted as WebSocket binary frames containing CBOR-encoded messages. No additional length prefix is required as WebSocket frames are self-delimiting. Mutual authentication requires an application-level handshake prior to capability exchange, as WebSockets do not natively support mutual TLS in all deployment environments. The Noise Protocol Framework IK handshake pattern is RECOMMENDED for this purpose.

C.4 Local Wireless Broadcast (Open Radio)

Discovery broadcasts are transmitted as unencrypted UDP datagrams to the local broadcast or multicast address. Each datagram contains exactly one CBOR-encoded message with no length prefix. Broadcasts MAY include the full discovery manifest or MAY use lightweight discovery (peer_id and personal_hash only). Session establishment (HANDSHAKE and CAPABILITY_EXCHANGE) occurs over a unicast connection initiated after discovery. Authentication MUST be established before CAPABILITY_EXCHANGE on the unicast channel.

Appendix D. Conformance Levels

Two conformance levels are defined.

D.1 Minimal Conforming Implementation

- A minimal conforming implementation MUST implement:
- CBOR wire format with deterministic encoding per §5.1 and §5.2

- All fields defined in §4.1 (top-level manifest fields), including protocol_version
- Personal hash computation over deterministic CBOR per §4.5
- The four-message capability exchange sequence defined in §7.2, including CONFIRM reconciliation
- Role-based intersection logic per §7.4
- NONE outcome handling — session MUST close on NONE (§7.6)
- Rebroadcast on all normative triggers (§8.1)
- The built-in UNRESTRICTED and DENIED classification tiers (§6.1)
- Sequence number replay detection per §4.1
- Glare resolution per §7.1.3
- core.heartbeat.liveness.1 per Appendix B.1

D.2 Full Conforming Implementation

- A full conforming implementation MUST implement everything in D.1, plus:
- Peer cache with auto-deny per §9
- Classification resolution and trust gate evaluation per §6
- Scope parameter enforcement (rate_limit, ttl, and any registered extensions)
- Parameter negotiation via CONFIRM accepted_params per §7.3
- Active set continuity during re-exchange per §8.4
- COSE credential verification per §5.4

Appendix E. Connection Archetypes — Worked Examples

The following examples use the four-message exchange sequence defined in §7.2. All messages are CBOR-encoded per §5; examples are shown in CBOR diagnostic notation for readability.

E.1 NONE / NONE Archetype

Neither peer offers capabilities the other can consume.

```
A -> B : OFFER { 1:1, 2:{1:1, 2:h'A1...', 3:42,
         4:[{1:"core.stream.relay.1", 2:1}],
         5:h'FF01...', 6:"sha-256"} }
B -> A : OFFER { 1:1, 2:{1:1, 2:h'B2...', 3:17,
         4:[{1:"core.stream.relay.1", 2:1}],
         5:h'FF02...', 6:"sha-256"} }
```

```
A computes intersection: PROVIDE + PROVIDE = no match
A -> B : CONFIRM { 1:2, 2:h'FF01...', 3:[] }

B computes intersection: PROVIDE + PROVIDE = no match
B -> A : CONFIRM { 1:2, 2:h'FF02...', 3:[] }

Both peers reconcile: [] ∩ [] = []
Both peers: NONE
A -> B : CLOSE { 1:3, 2:h'FF01...', 3:1 }
Both peers store: { peer_id, last_hash, last_outcome: NONE }
```

E.2 ASYMMETRIC Archetype

A provides a stream capability; B consumes it.

```
A -> B : OFFER { 1:1, 2:{..., 4:[{1:"core.stream.relay.1", 2:1,
         5:{1:0, 2:3600}}], ...} }
B -> A : OFFER { 1:1, 2:{..., 4:[{1:"core.stream.relay.1", 2:2}], ...} }

A computes intersection: PROVIDE + CONSUME = match
A evaluates trust gate for peer_B -> ALLOW
A -> B : CONFIRM { 1:2, 2:h'...', 3:["core.stream.relay.1"] }

B computes intersection: CONSUME + PROVIDE = match
B evaluates trust gate for peer_A -> ALLOW
B -> A : CONFIRM { 1:2, 2:h'...', 3:["core.stream.relay.1"] }

Reconciliation: ["core.stream.relay.1"] ∩ ["core.stream.relay.1"]
Both peers: ACTIVE (1 capability)
```

E.3 ASYMMETRIC with Trust Disagreement

Both peers compute a match, but their trust gates produce different results.

```
A -> B : OFFER { ..., 4:[
         {1:"core.stream.relay.1", 2:1},
         {1:"core.feed.paginated.1", 2:1}], ... }
B -> A : OFFER { ..., 4:[
         {1:"core.stream.relay.1", 2:2},
         {1:"core.feed.paginated.1", 2:2}], ... }

A trust gates: both ALLOW
A -> B : CONFIRM { 1:2, 2:h'...', 3:["core.feed.paginated.1",
         "core.stream.relay.1"] }
```

```
B trust gates: stream ALLOW, feed DENY
B -> A : CONFIRM { 1:2, 2:h'...', 3:["core.stream.relay.1"] }

Reconciliation: intersection = ["core.stream.relay.1"]
Both peers: ACTIVE (1 capability)
```

E.4 MUTUAL Archetype with Parameter Negotiation

Both peers provide and consume capabilities simultaneously. Scope parameters are negotiated.

```
A -> B : OFFER { ..., 4:[
         {1:"core.stream.relay.1", 2:1, 5:{2:7200}},
         {1:"core.feed.paginated.1", 2:2, 5:{1:100, 2:3600}},
         {1:"core.heartbeat.liveness.1", 2:3, 3:true}], ... }

B -> A : OFFER { ..., 4:[
         {1:"core.stream.relay.1", 2:2, 5:{2:3600}},
         {1:"core.feed.paginated.1", 2:1, 5:{1:50, 2:7200}},
         {1:"core.heartbeat.liveness.1", 2:3, 3:true}], ... }

A computes intersection: 3 matches
A -> B : CONFIRM { 1:2, 2:h'...',
         3:["core.feed.paginated.1", "core.heartbeat.liveness.1",
            "core.stream.relay.1"],
         4:{"core.feed.paginated.1": {1:50, 2:3600},
            "core.stream.relay.1": {2:3600}} }

B mirrors computation
B -> A : CONFIRM { 1:2, 2:h'...',
         3:["core.feed.paginated.1", "core.heartbeat.liveness.1",
            "core.stream.relay.1"],
         4:{"core.feed.paginated.1": {1:50, 2:3600},
            "core.stream.relay.1": {2:3600}} }

Reconciliation: 3-way match.
  core.feed.paginated.1: rate_limit=min(100,50)=50, ttl=min(3600,7200)=3600
  core.stream.relay.1:   ttl=min(7200,3600)=3600
Both peers: ACTIVE (3 capabilities, reconciled params)
```

E.5 Rebroadcast and Re-evaluation

A is ACTIVE with B. A removes core.stream.relay.1 from its capability list.

```
A recomputes personal_hash (hash_A2, sequence_num++)
A -> B : OFFER { 1:1, 2:{..., 3:43, 4:[
```

```
         {1:"core.feed.paginated.1", 2:2},
         {1:"core.heartbeat.liveness.1", 2:3, 3:true}],
         5:h'hash_A2', ...} }

B observes hash_A2 != hash_A -> re-enters CAPABILITY_EXCHANGE
Note: existing active set (3 capabilities) remains operational

B -> A : OFFER { ..., 4:[...], ... }

A -> B : CONFIRM { 1:2, 2:h'hash_A2',
         3:["core.feed.paginated.1", "core.heartbeat.liveness.1"] }
B -> A : CONFIRM { 1:2, 2:h'...',
         3:["core.feed.paginated.1", "core.heartbeat.liveness.1"] }

Reconciliation: 2-way match.
core.stream.relay.1: deactivated
core.feed.paginated.1: not interrupted
core.heartbeat.liveness.1: not interrupted
Session remains ACTIVE with 2 capabilities.
```

E.6 Auto-Deny

A has previously completed a NONE exchange with B. B reappears with same hash.

```
B broadcast received by A (CBOR: {2:h'B2...', 5:h'FF02...'})
A checks peer cache: peer_id_B found
  last_hash == current hash_B : true
  last_outcome == NONE        : true

A records outcome: DENIED
A does NOT initiate handshake

--- Later: B updates its capabilities, rebroadcasts with hash_B2 ---

A checks peer cache: peer_id_B found
  last_hash (hash_B) != current hash (hash_B2) : true
A transitions peer_B from DENIED to PEER_VISIBLE
A initiates new handshake -> CAPABILITY_EXCHANGE
```

Authors

Ivy Darling

This document is a work in progress. Author contact information to be added prior to publication.